

Лабораторная работа №2

Введение в объектно-ориентированное программирование. Классы и объекты

*Задания лабораторной работы основаны на курсе «Backend-разработчик» АНО ДПО
«Образовательные технологии Яндекса»*

1. Что необходимо знать для выполнения работы

В работе пригодится понимание разницы между классом и объектом класса, а также знание того, что такое:

- инициализатор класса;
- метод объекта;
- атрибут класса;
- атрибут объекта.

2. Задание №1. Колония бактерий

Исходный код к данному заданию находится в файле `main.py`.

2.1. Условие

В биологической лаборатории выводят премиальную породу очень полезных бактерий. Отдельные колонии бактерий выращивают в автоматизированных чашках Петри.

Но бактерии вырвались на свободу и устроили короткое замыкание. Система управления сгорела, программа была уничтожена. Спасать ситуацию выпало именно вам: надо восстановить программу.

2.2. Как должна работать программа

При создании новой колонии в программе создаётся объект класса `BacteriaProducer` и устанавливается предельная численность популяции: в инициализатор класса передаётся параметр `max_bacteria`.

Текущее количество бактерий в колонии хранится в атрибуте объекта `current_bacteria_count`.

Помимо инициализатора у экземпляров класса `BacteriaProducer` есть два метода:

- при вызове метода `delete()` должна удаляться одна бактерия из популяции: счётчик `current_bacteria_count` должен уменьшиться на 1;
- при вызове метода `create()` в популяцию должна добавляться одна бактерия: счётчик `current_bacteria_count` должен увеличиться на 1.

Значение счётчика `current_bacteria_count` не должно превышать значение `max_bacteria` и не должно быть меньше 0.

2.3. Что нужно сделать

1. В инициализаторе класса опишите два атрибута:
 - атрибут `max_bacteria` должен быть равен значению аргумента `max_bacteria`, переданного в инициализатор класса при создании объекта;
 - атрибут `current_bacteria_count` должен хранить количество бактерий в каждый текущий момент времени. По умолчанию этот атрибут должен быть равен 0.
2. Опишите метод `create()`, который отвечает за добавление бактерии в колонию.

При вызове этого метода счётчик бактерий `current_bacteria_count` должен увеличиться на единицу.

Если в момент вызова этого метода количество бактерий в популяции `current_bacteria_count` равно максимально возможному количеству бактерий в колонии `max_bacteria` — новую бактерию добавлять не следует; программа должна напечатать сообщение об ошибке:

```
Нет места под новую бактерию
```

В ином случае увеличьте счётчик бактерий на единицу и напечатайте сообщение:

```
Добавлена одна бактерия. Бактерий в колонии: <число>
```

3. Опишите метод `delete()`, который при каждом вызове удаляет из колонии по одной бактерии.

Если в момент вызова этого метода количество бактерий в популяции `current_bacteria_count` равно нулю — бактерию удалить невозможно. Программа должна напечатать сообщение об ошибке:

```
В популяции нет бактерий, удалять нечего
```

В ином случае уменьшите счётчик бактерий на единицу и напечатайте сообщение:

```
Одна бактерия удалена. Бактерий в колонии: <число>
```

3. Задание №2. Корзина для грибов

Исходный код к данному заданию находится в файле `task2.py`.

3.1. Условие

Разработчик начал писать приложение для грибников, но наступила зима — и ему стало не до грибов. Вам предстоит закончить работу над приложением.

Идея приложения такова: создаётся программная «корзина для грибов» (**объект** класса `MushroomsCollector`), и каждый найденный в лесу гриб добавляется в эту корзину (учитывается в программе). Если гриб несъедобен — программа должна запретить добавление этого гриба в корзину.

Может быть создано несколько таких «программных корзин».

3.2. Что нужно сделать

В прекоде есть класс `MushroomsCollector`, он отвечает за проверку и сохранение грибов. Допишите этот класс и исправьте ошибки.

1. Сейчас `mushrooms` объявлен как атрибут класса, поэтому все корзины делят один общий список. Исправьте код так, чтобы каждая корзина хранила собственный список грибов.
2. Метод `is_poisonous()` должен принять на вход название гриба и проверить, не ядовит ли гриб. Сейчас этот метод работает с ошибкой и признаёт все грибы ядовитыми. Исправьте этот метод так, чтобы он нормально выполнял свою задачу.
3. Метод `add_mushroom()` должен принять на вход название гриба и, если гриб не ядовит (если метод `is_poisonous()` вернул `False`), сохранить его в список `mushrooms`.

Если же метод `is_poisonous()` вернул `True` — программа должна напечатать сообщение об ошибке:

```
| Нельзя добавить ядовитый гриб
```

4. Реализуйте магический метод `__str__`, который выводит через запятую и пробел элементы списка `mushrooms` конкретного объекта класса `MushroomsCollector`. Например:

```
| белый, рыжик, груздь, лисичка
```